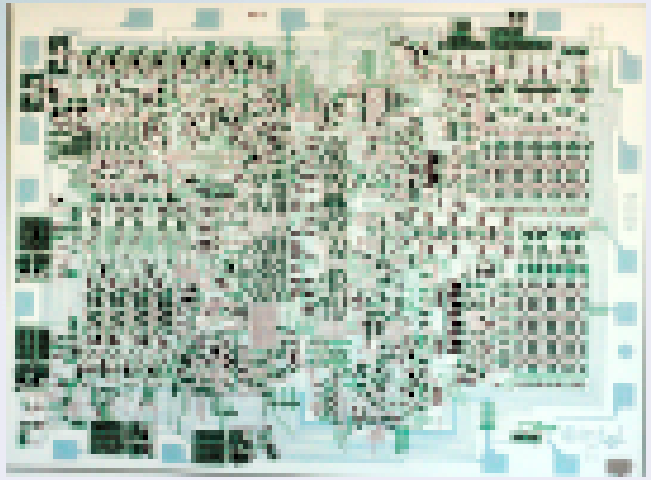


Lecture 13a:

LC3 C -> LC3 Assembly

CSCI11: **Computer Architecture and Organization**

Deep Dives into LC3 Assembly from the LC3 C Compiler

Note:

The C compiler was written by students at Georgia Tech. I've tested it extensively and have made modifications either to fix bugs or to improve output.

That said, its not a polished C compiler, nor has it seen extensive use. It is valuable in beginning to understand how compilation works and how to convert a HLL into an ISA.

This deck was created with extensive help from Claude AI.

Abs: From C to LC3 Assembly.

Write a program that computes the **absolute value** of the integer `–7` and stores (or prints) the result.

The C Source Code

```c
int main()
{
    int number = -7;
    int abs = 0;
    if (number >= 0)
    {
        abs = number;
    }
    else
    {
        abs = -number;
    }
}
```

The Bootstrap — Where Execution Begins

LC3 starts every program at address `x3000`. The compiler emits a small bootstrap that prepares the stack, calls `main`, and halts.

```
.ORIG x3000
LD R6, USER_STACK   ; R6 = xFDFF (top of user stack)
ADD R5, R6, #-1     ; R5 = xFDFE (initial frame pointer)
JSR main            ; call main; R7 ← return address
HALT                ; program ends here
```

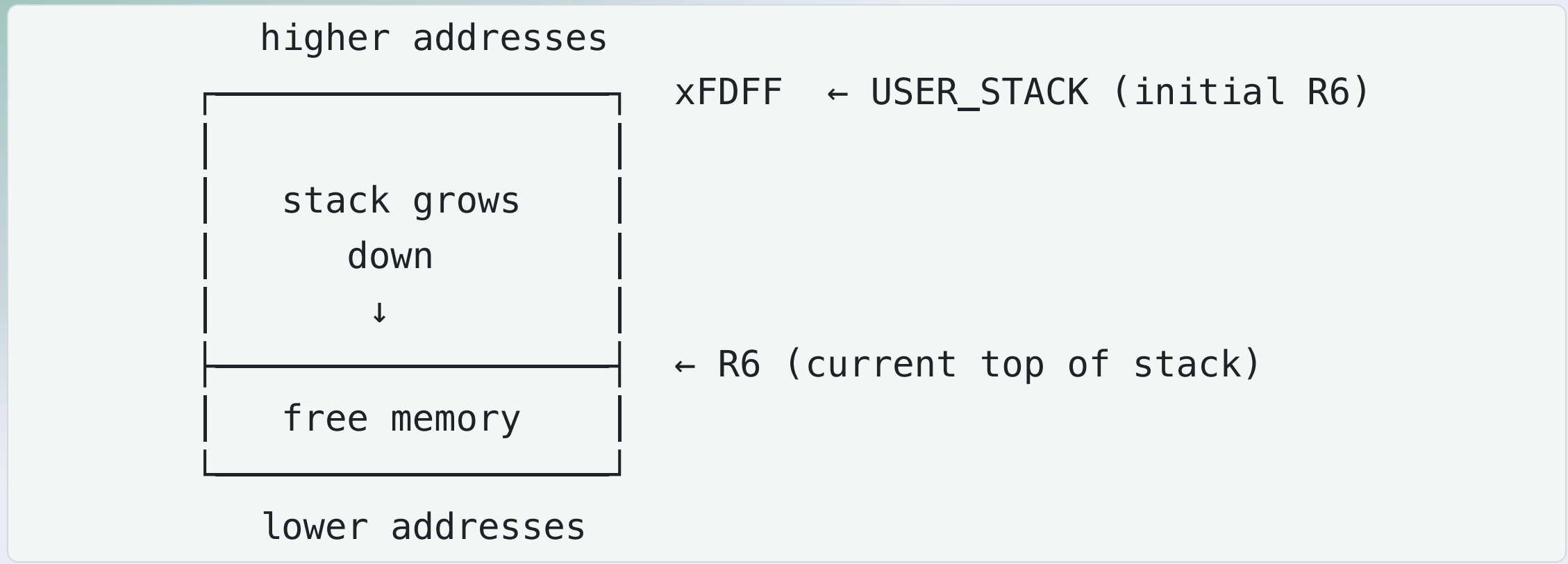
Two ideas to notice:

- **`main` is called like any other function.** `JSR` saves the return address in R7 and jumps. When `main` finishes, it executes `RET` and control returns *to the line after the JSR* — the `HALT`.
- **`HALT` is the single exit point.** No matter how many `return` statements `main` contains, every one of them eventually funnels back through `RET` to this `HALT`. Symmetric with every other function in the program.

R5 and R6 — Two Pointers, Two Jobs

The bootstrap initializes two registers. Why two?

- **R6 is the stack pointer (SP).** It marks where free stack memory begins. It moves every time the program reserves or releases a stack slot.
- **R5 is the frame pointer (FP).** Once a function starts, R5 is fixed for the duration of that function. Local variables are always accessed at fixed offsets from R5: `R5+0`, `R5−1`, `R5−2`, ...



If we used only one pointer, every push or pop would shift the offset of every local variable. With the FP/SP split, the *frame pointer* gives stable compile-time offsets while the *stack pointer* tracks live usage for calls, traps, and temporaries.

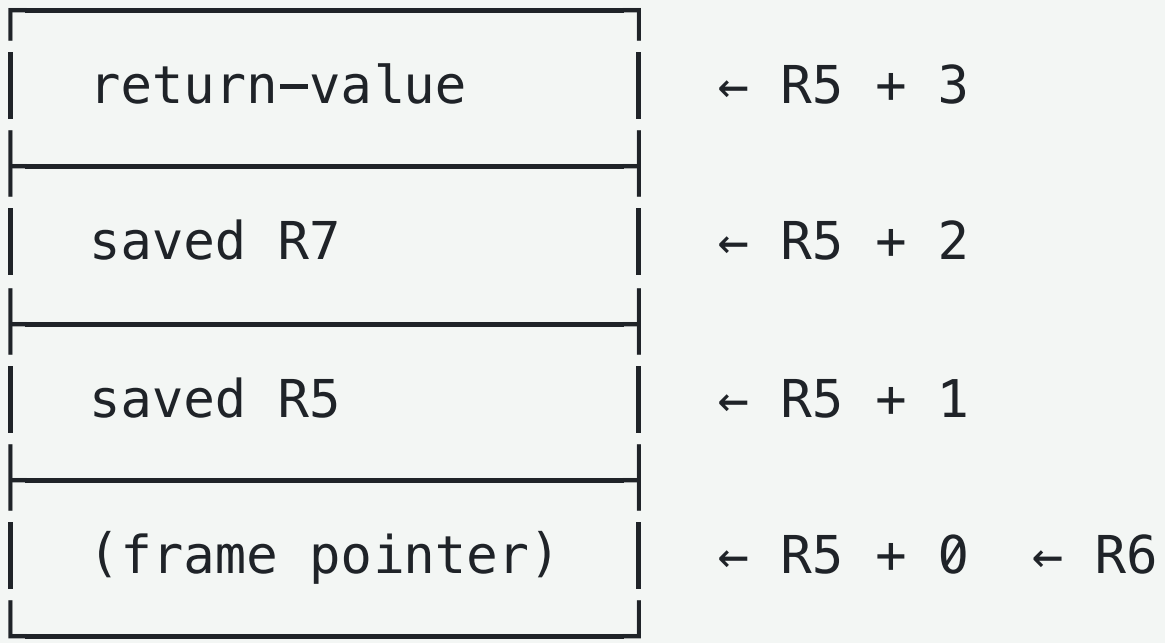
main's Prologue — Callee Setup

When `main` is entered, it builds its stack frame the same way every function does. This sequence is called the **prologue** or **callee setup**.

```
main
; callee setup:
    ADD R6, R6, #-1   ; allocate spot for return value
    ADD R6, R6, #-1
    STR R7, R6, #0    ; save caller R7 (return address)
    ADD R6, R6, #-1
    STR R5, R6, #0    ; save caller R5 (caller frame pointer)
    ADD R5, R6, #-1   ; set frame pointer for main
```

The prologue saves three things on the stack:

1. **A return-value slot** — empty space the function will fill before returning.
2. **R7** — the return address that `JSR` placed here. Saved because the function might call other functions, which would overwrite R7.
3. **R5** — the caller's frame pointer. Saved because we are about to overwrite R5 with our own.



After the prologue, R5 points at the spot just above where the first local will land. Locals are addressed at `R5+0, R5−1, R5−2, ...`; the saved registers and return slot are at the positive offsets above R5.

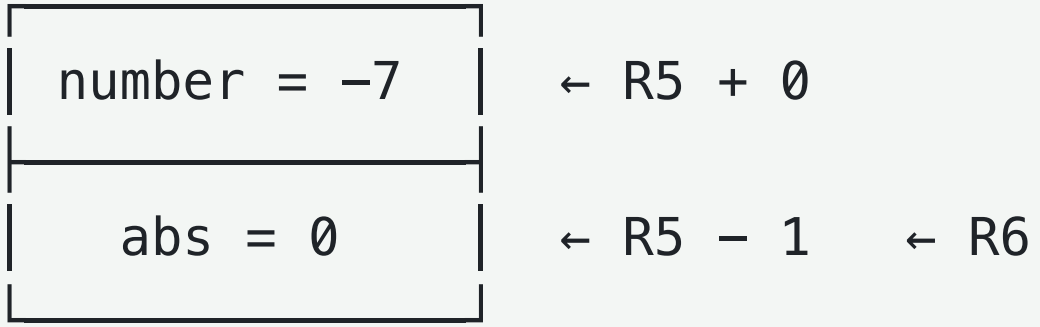
Local Variables and Negative Literals

The body declares two ints and initializes by decrementing R6 to reserve a slot, compute initializer, store it through R5 at the slot's known offset.

```
; function body:
    ADD R6, R6, #-1       ; reserve slot for "number"
    AND R0, R0, #0        ; R0 = 0
    ADD R0, R0, #7        ; R0 = 7
    NOT R0, R0            ; R0 = ~7 = 0xFFF8
    ADD R0, R0, #1        ; R0 = -7 = 0xFFF9
    STR R0, R5, #0        ; number = -7
    ADD R6, R6, #-1       ; reserve slot for "abs"
    AND R0, R0, #0        ; R0 = 0
    STR R0, R5, #-1       ; abs = 0
```

Superfluous Step - not true, however, not worth fixing

This point is wrong, both in compiler and Claude:
LC3's `ADD` can only load small positive immediates (range −16 to +15), and there is no `NEG` or `SUB` instruction. To produce `−7`, the compiler builds it: load `+7`, then **two's-complement negate** by `NOT` followed by `ADD …, #1`.



The LC3 can handle negative immediate values.

Demo code

Sidebar — How an Earlier Version Did Branching

A prior release of the compiler used a "universal branch" pattern:

- always emit `BRnz` to skip the if-body, and rewrite the *expression* to fit.
- `>=`, that meant computing `(number + 1) > 0` — algebraically equivalent to `number ≥ 0`.

Worked, but

- cost an extra `ADD …, #1` per `>=`/`<=` comparison
- had a subtle bug: on 16-bit LC3, when `a − b == +32767`, the boundary `+1` wrapped to `−32768` and silently flipped the result.

Evaluating `number >= 0` — Branch-Flag Selection

The C condition is `number >= 0`.

LC3's `BR` instruction encodes three independent flag bits — N, Z, P:

C Operator	True when result is	Branch on FALSE
`<`	negative	`BRzp` (zero or positive)
`>`	positive	`BRnz` (negative or zero)
`<=`	negative or zero	`BRp` (positive)
`>=`	zero or positive	`BRn` (negative)
`==`	zero	`BRnp` (negative or positive)
`!=`	negative or positive	`BRz` (zero)

Code

The compiler emits a single subtraction `left - right`. After the subtraction, pick the branch from the table — for `>=`, that is `BRn` (BR only when the difference is negative, i.e., when `number < 0`).

```
    AND R0, R0, #0        ; R0 = 0     (the literal 0 on the right of >=)
    LDR R1, R5, #0        ; R1 = number
    NOT R0, R0            ; \  two's-complement negate the right operand
    ADD R0, R0, #1        ; /  R0 = -(0) = 0
    ADD R1, R1, R0        ; R1 = number - 0;  ADD sets NZP as a side effect
    BRn  main_if_0        ; if number < 0 (>= is false), jump to else
```

If-Else Control Flow

```
    BRn  main_if_0           ; if false (number < 0), jump to else

    ; --- if-body: abs = number ---
    LDR R0, R5, #0
    STR R0, R5, #-1
    BR   main_if_0_end       ; unconditional skip past else

main_if_0
    ; --- else-body: abs = -number ---
    LDR R0, R5, #0
    NOT R0, R0
    ADD R0, R0, #1           ; 2's complement pattern
    STR R0, R5, #-1

main_if_0_end
```

main's Teardown — Returning to the Bootstrap

After the if-else merges, `main` falls through to its **teardown**, which reverses the prologue.

```
main_teardown
    ADD R6, R5, #1      ; pop local variables (R6 ← above the locals)
    LDR R5, R6, #0      ; pop frame pointer (restore caller's R5)
    ADD R6, R6, #1
    LDR R7, R6, #0      ; pop return address
    ADD R6, R6, #1
    RET                 ; jump to R7  →  back to the bootstrap's HALT
```

`ADD R6, R5, #1` is the magic line: it discards every local variable in one instruction by snapping R6 back to where R5 sits (just above the locals). From there, the prologue's three pushes are undone in reverse order: pop R5, pop R7, then `RET`.

`RET` is `JMP R7`, so control returns to the instruction right after `JSR main` in the bootstrap — which is `HALT`. That is the **single exit point** of the program. Whether `main` had zero, one, or many `return` statements, all of them route through `BR main_teardown → RET → HALT`.

Putting It All Together (*Full Lifecycle of Program*)

```
1. Bootstrap          .ORIG x3000
                      LD R6, USER_STACK        ┐  set up SP and FP
                      ADD R5, R6, #-1          ┘
                      JSR main                 ── call main
                      HALT                     ── single program exit

2. main prologue      push return slot, R7, R5, set new R5
                      (so locals can be addressed via R5)

3. main body          declare and initialize "number" and "abs"
                      evaluate "number >= 0" via subtraction + BRn
                      branch into if-body or else-body
                      assign "abs"

4. main teardown      pop locals, restore R5, restore R7
                      RET back to the bootstrap's HALT
```

Three techniques recur throughout the compiler's output and are worth memorizing:

- **`NOT R, R` followed by `ADD R, R, #1`** — two's-complement negation. Used for negative literals, unary minus, and as the back half of every subtraction.
- **`ADD R6, R6, #-1` followed by `STR …, R6, #0`** — the "push" idiom. The reverse (`LDR …, R6, #0` then `ADD R6, R6, #1`) is "pop."
- **Branch-flag selection per operator** — every C comparison maps to exactly one LC3 `BR` variant whose flags are the operator's negation (see the table earlier in the deck). The subtraction sets NZP as a side effect, so no separate compare is needed.

Once these patterns are familiar, most LC3 output from this compiler reads almost as easily as the C it came from.

Functions: The Calling Convention in LC3

A walk through how multiple functions interact at the assembly level: passing arguments, returning values, and the contract between caller and callee.

This deck builds on the `abs` background — the program-entry handshake, R5/R6 split, prologue, teardown, and two's-complement negation are introduced there and are reused unchanged here.

The C Source — Three Functions, Two Calls

```c
int add_2(int a, int b)
{
    int c = a + b;
    return c;
}

int sub_2(int a, int b)
{
    int c = a - b;
    return c;
}

void main()
{
    int a = 21;
    int b = 33;
    int c = add_2(a, b);
    int d = sub_2(a, b);
    return;
}
```

Overall

- Three functions
- Four locals in `main`
- Two function calls
- Each callee has the **same shape** as `main` had in the `abs` example: prologue, body, teardown, `RET`.

What's new in this example is the *handshake* between functions — how arguments arrive, how a return value gets back, and who is responsible for cleaning up which part of the stack.

The Caller–Callee Contract

A function call is a four-step handshake.

Step	Who	What
1. Push arguments	**Caller**	Push args right-to-left, then `JSR`
2. Build frame	**Callee**	Reserve return slot, save R7 & R5, set new R5
3. Execute body	**Callee**	Use parameters and locals via `R5 ± offset`
4. Write return value	**Callee**	Store result into the return slot at `R5 + 3`
5. Tear down	**Callee**	Pop locals, restore R5 & R7, `RET`
6. Read result, pop args	**Caller**	Read return value at `R6 + 0`, then pop

Pushing Arguments — Right-to-Left

Before each `JSR`, the caller pushes the function's arguments onto the stack. The convention is **right-to-left**: the *last* parameter is pushed *first*, so the *first* parameter ends up *on top* of the stack.

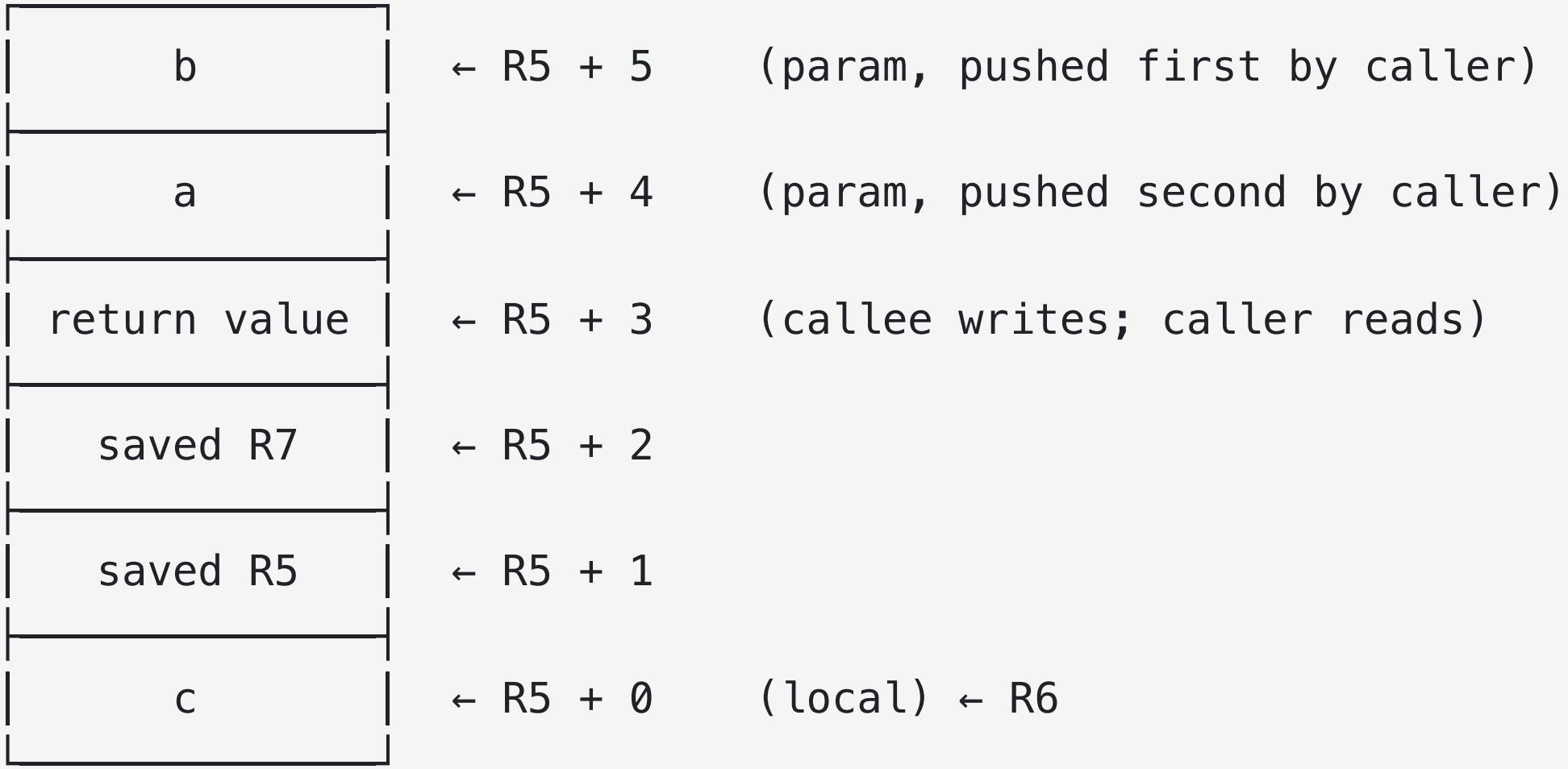
```
; in main, calling add_2(a, b):

    LDR R0, R5, #-1   ; load b (main's local)
    ADD R6, R6, #-1
    STR R0, R6, #0    ; push b first

    LDR R0, R5, #0    ; load a (main's local)
    ADD R6, R6, #-1
    STR R0, R6, #0    ; push a second (now on top)

    JSR add_2         ; R7 ← return address, jump
```

After the two pushes and the `JSR`, the stack from `add_2`'s perspective looks like:

```
...
[   b   ]   ← pushed first  (further from new R5)
[   a   ]   ← pushed second (closer to new R5)   ← R6
```

Why right-to-left? It makes parameter offsets predictable from the callee's side. After `add_2`'s prologue saves R7 and R5 *below* the arguments, the layout becomes deterministic: parameter *N* always lands at `R5 + (3 + N)`. The first parameter is at `R5 + 4`, the second at `R5 + 5`, regardless of how many arguments there are.

The Stack Frame from the Callee's View (*After `add_2` runs its prologue*)



Two halves divided by R5:

- **Above R5** (positive offsets): things provided *by* or *for* the caller — parameters and the return slot. These are part of the **call interface**.
- **Below R5** (zero or negative offsets): things owned entirely by the callee — its local variables.

The compiler uses this split when it generates code:

```
; int c = a + b;
LDR R0, R5, #4      ; load parameter "a" (R5 + 4)
LDR R1, R5, #5      ; load parameter "b" (R5 + 5)
ADD R0, R0, R1
STR R0, R5, #0      ; store local "c"   (R5 + 0)
```

Notice the assembly has no idea whether `a` and `b` were pushed by `main`, by `sub_2`, or by some other caller. The contract guarantees the layout, and that's all the *codegen* needs to know.

Returning a Value — Writing to `R5 + 3`

The C statement `return c;` becomes two instructions: store the result into the return-value slot, then branch to teardown.

```
LDR R0, R5, #0          ; load local "c"
STR R0, R5, #3          ; write return value, always R5 + 3
BR  add_2_teardown      ; single-exit funneling
```

Why `R5 + 3`? Look back at the frame diagram. The prologue (in `abs` Slide 4) reserves *one slot above the saved R7 and R5* — and `R5 + 3` is exactly that slot. It is the same offset for every function, regardless of how many parameters there are or how big the local block is.

This keeps the rule simple: **return values always go to R5 + 3.** The compiler hard-codes that offset; the caller hard-codes that offset too (read further on).

`BR add_2_teardown` is the single-exit funnel from `abs` Slide 8 — every `return` statement in the function routes through one teardown sequence rather than emitting cleanup inline at each return point.

Caller Cleanup — Read First, Then Pop

After `RET` brings control back to the caller, the stack still holds three caller-visible slots: the return value, then the two arguments below it.

```
...
[   b   ]
[   a   ]
[ retval ]   ← R6
```

The caller's cleanup code reads the return value first, then pops everything in one shot:

```
JSR add_2
LDR R0, R6, #0        ; read return value (R6 still points to it)
ADD R6, R6, #1        ; pop the return slot
ADD R6, R6, #2        ; pop the two arguments
STR R0, R5, #-2       ; store result into main's local "c"
```

Order matters. The return value's lifetime ends the moment R6 advances past its slot — once popped, that memory is fair game for the next call or trap. Two cardinal rules:

- **Read before popping.** `LDR R0, R6, #0` must come before `ADD R6, R6, #1`.
- **The caller pops the arguments, not the callee.** This is "caller cleanup," the LC3 textbook convention. It lets each caller decide what to do with the args (most just discard them).

Subtraction in `sub_2` — Reusing the Negation Idiom

`sub_2` does `a - b`. LC3 has no `SUB` instruction, so the compiler builds it from the parts it has:

```
; int c = a - b;
LDR R0, R5, #4      ; R0 = a
LDR R1, R5, #5      ; R1 = b
NOT R1, R1          ; \ two's-complement negate b:
ADD R1, R1, #1      ; /  R1 = -b
ADD R0, R0, R1      ; R0 = a + (-b) = a - b
STR R0, R5, #0      ; store local "c"
```

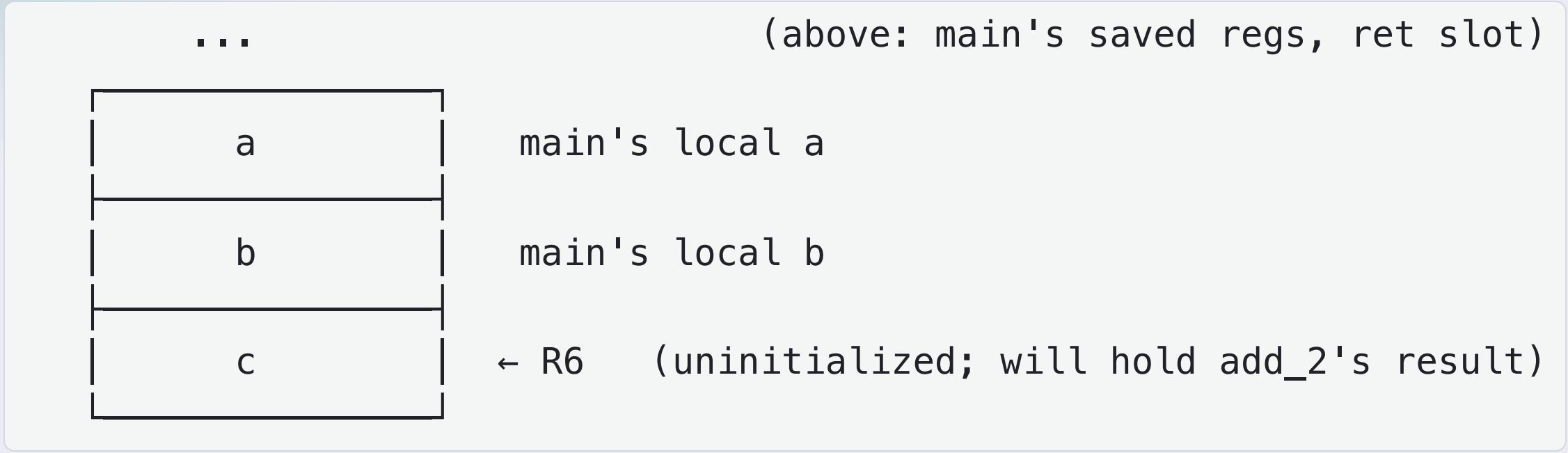
This is the same `NOT / ADD #1` idiom from `abs` Slide 5, now serving as the back half of subtraction: `a - b ≡ a + (-b)`. The same two instructions also appear inside the `>=` evaluation in `abs` Slide 6, and inside the unary minus `-number` in `abs` Slide 7. One pattern, three contexts — once recognized, it becomes invisible scaffolding around the actual logic.

A Full Call Trace — `int c = add_2(a, b)`

Tie everything together by watching the stack at four key moments during the call. Slot labels are semantic (what each slot *means*), not numeric values — that keeps it easy to see who owns what.

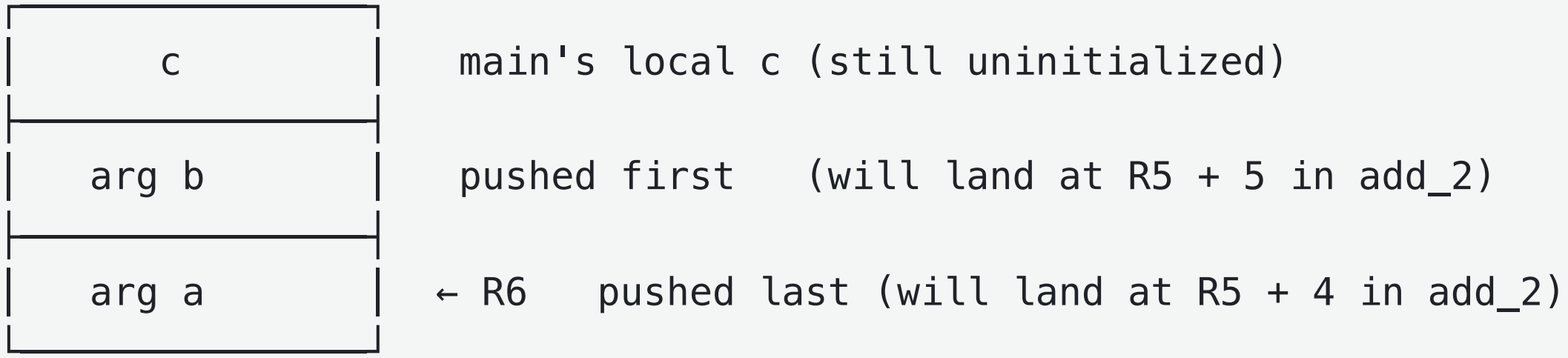
Snapshot 1 — In `main`, after allocating `c`, ready to push arguments

By the time `main` is about to call `add_2`, it has already allocated three locals: `a`, `b`, and `c` (the slot for `c` is reserved *before* the arguments are pushed).

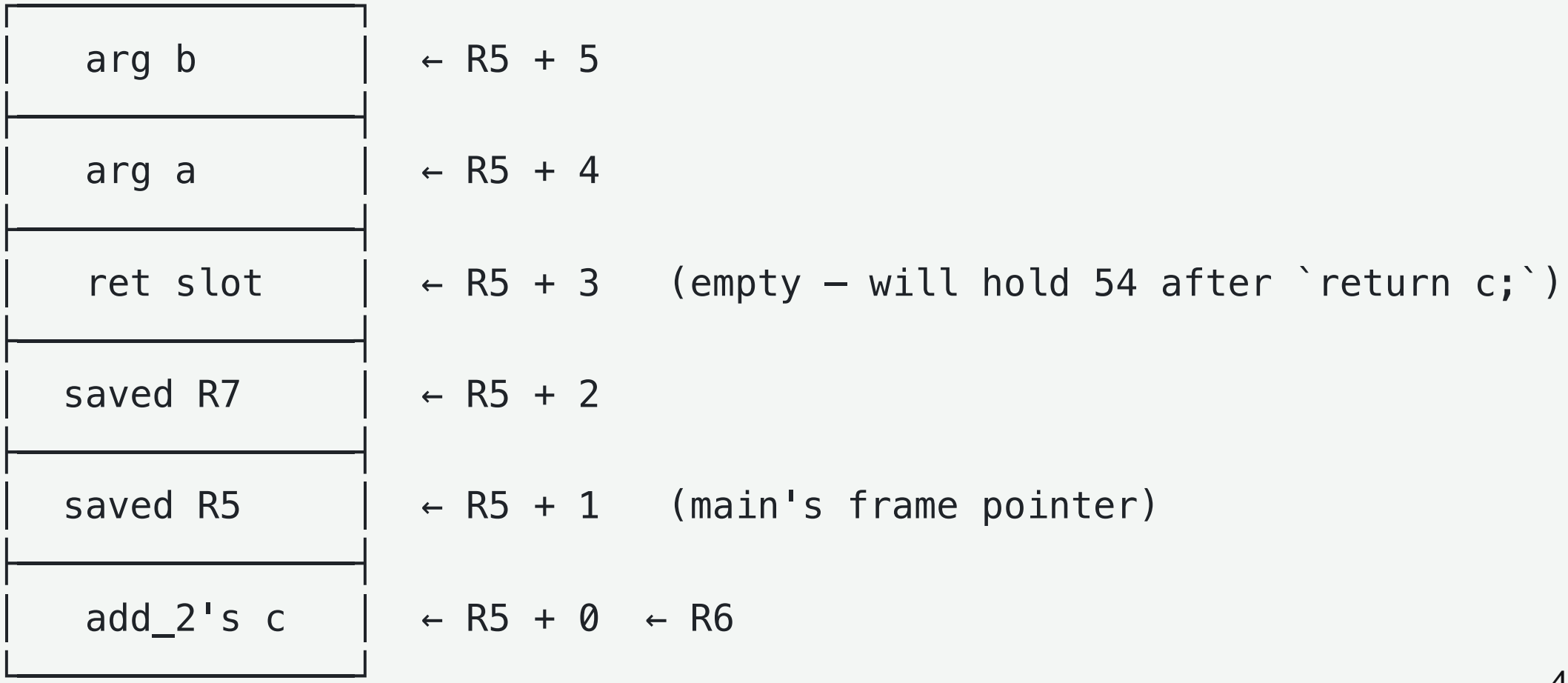


Snapshot 2 — After pushing arguments, immediately before `JSR add_2`

`main` loads its local `b` and pushes a copy as the second argument; loads its local `a` and pushes a copy as the first argument (top of stack). The original `a` and `b` *stay where they are* — only copies are pushed.

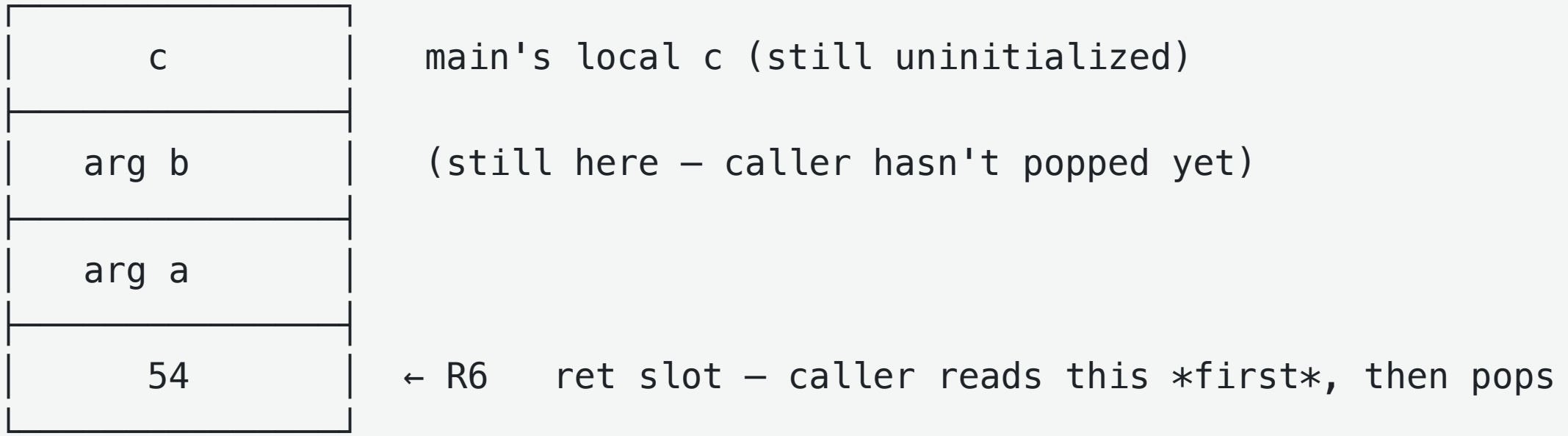


Snapshot 3 — Inside `add_2`, after prologue and allocating its own local `c`



Snapshot 4 — Back in `main` after `RET`, just before reading the return value

The teardown popped `add_2`'s local, restored R5 and R7, then `RET` jumped back into `main`. R6 sits exactly on the return slot (which now holds 54).



`main` then runs `LDR R0, R6, #0` (read 54), `ADD R6, R6, #1` (pop ret slot), `ADD R6, R6, #2` (pop both args), and `STR R0, R5, #-2` (store 54 into main's local `c`). The stack is back to Snapshot 1's shape — but `c` is now initialized.

Every region of the stack is the responsibility of *exactly one* party at any moment.

- Arguments are written by the caller and read by the callee
- Return slot is written by the callee and read by the caller
- Locals are private to whichever function R5 is currently pointing into

The contract is what makes all of this work — and what lets the compiler generate caller code and callee code completely independently.